



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий
Кафедра математического обеспечения и стандартизации информационных
технологий

ОТЧЕТ

ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 4

по дисциплине

«Тестирование и верификация программного обеспечения»

Выполнили студенты группы *ИКБО-74-23 Команда ЗавозWW*

Кузьмин И.В.

Тыцкий Б.С.

Принял

Ильичев Г.П.

Практическая

«05» ноября 2025 г.

работа выполнена

«Зачтено»

«__» 2025 г.

Москва 2025

Техническое задание проекта анализатора кода

1. Аннотация

Краткое описание: ознакомиться с основными принципами и методами использования статических и динамических анализаторов кода для раннего выявления ошибок и потенциальных уязвимостей, что позволит повысить качество, безопасность и надёжность программного обеспечения.

2. Введение

- Основание: практическое задание по дисциплине «Тестирование и верификация программного обеспечения».
- Цель: закрепление навыков разработки, тестирования и анализа качества кода на двух учебных проектах разной технологической природы.
- Область применения: учебный процесс, демонстрация методологий тестирования, сопоставление статического и динамического анализа.

3. Техническое задание

3.1 Изучить теоретические основы статического и динамического анализа кода.

3.2 Ознакомиться с популярными инструментами статического анализа (например, ESLint, Pylint, Checkmarx, SonarQube, FindBugs, TSLint, Cppcheck) и динамического анализа (например, Valgrind, DynamoRIO, Java VisualVM, Burp Suite, OWASP ZAP)

3.3 Применить выбранные анализаторы к ранее разработанным учебным проектам на разных языках программирования.

3.4 Провести анализ исходного кода до и после внесения целенаправленных ошибок, оценить адекватность обнаружения дефектов.

3.5 Сформировать детальный отчёт с критическим анализом результатов, выводами о преимуществах и ограничениях каждого подхода.

4. Результаты

1. Титульный лист, включающий в себя наименование работы, автора, дату выполнения, название учебной дисциплины.
2. Краткое описание каждого проекта, используемых анализаторов и примененных методов анализа.
3. Таблицы или графики, отображающие количество и тип обнаруженных ошибок до и после внесения намеренных дефектов.
4. Сравнительный анализ результатов до и после внесения ошибок.
5. Сравнение отчетов статического и динамического анализа, обсуждение ложных срабатываний, пропущенных ошибок и возможных причин таких результатов.
6. Обоснованные выводы о целесообразности применения статического и динамического анализа в различных условиях, рекомендации по выбору инструментов для разных типов проектов и интеграции анализа в процессы разработки.
7. Приложения (при необходимости): дополнительные материалы (коды, логи тестирования, скриншоты и т. д.).

5. Выводы

- Было подготовлено 2 программных продукта для применения анализаторов кода на практике;
- Продемонстрировано выполнение всех шагов динамического тестирования;
- Продемонстрировано выполнение всех шагов статического тестирования;
- Написан полноценный отчет в котором расписаны обоснованные выводы о целесообразности применения статического и динамического анализа в различных условиях, рекомендации по выбору инструментов для разных типов проектов и интеграции анализа в процессы разработки;

6. Функционал обоих приложений

6.1 Функционал и код рекомендателя фильмов

```
class Recommender:
    def __init__(self):
        self.catalog = {
            "Терминатор 2": "Научная фантастика",
            "Чужой": "Научная фантастика",
            "The Matrix": "Научная фантастика",
            "Бойцовский клуб": "Драма",
            "Форрест гамп": "Драма",
            "Криминальное чтиво": "Драма"
        }
        self.ratings = {}
        self.popular = ["Топ-10 популярных фильмов"]

    def add_rating(self, user, movie, rating):
        if rating < 0 or rating > 5:
            raise ValueError("Оценка должна быть от 0 до 5")
        if movie not in self.catalog:
            raise ValueError("Фильм отсутствует в каталоге")
        self.ratings.setdefault(user, {})[movie] = rating

    def get_recommendations(self, user):
        if user not in self.ratings or not self.ratings[user]:
            return self.popular

        preferred_genres = {
            self.catalog[movie]
            for movie, rating in self.ratings[user].items()
            if rating >= 4
        }

        recs = [
            movie for movie, genre in self.catalog.items()
            if genre in preferred_genres and movie not in
self.ratings[user]
        ]
        return recs if recs else self.popular

    def filter_by_genre(self, genre):
        return [movie for movie, g in self.catalog.items() if g.lower()
== genre.lower()]
```

```
def main():
    rec = Recommender()
    user = input("Введите имя пользователя: ")

    while True:
        print("\nМеню:")
        print("1. Поставить оценку фильму")
        print("2. Получить рекомендации")
        print("3. Фильтровать фильмы по жанру")

        choice = input("Выберите действие: ")

        if choice == "1":
            print("Доступные фильмы:", ", ".join(rec.catalog.keys()))
            movie = input("Введите название фильма: ")
            try:
                rating = float(input("Введите оценку (0-5): "))
                rec.add_rating(user, movie, rating)
                print(f"Оценка {rating} для фильма '{movie}'  
сохранена.")
            except Exception as e:
                print("Ошибка:", e)

        elif choice == "2":
            recs = rec.get_recommendations(user)
            print("Рекомендации:", ", ".join(recs))

        elif choice == "3":
            genre = input("Введите жанр (например, Научная фантастика  
или Драма): ")
            movies = rec.filter_by_genre(genre)
            if movies:
                print("Фильмы жанра", genre, ":", ", ".join(movies))
            else:
                print("Фильмы данного жанра не найдены.")

        else:
            print("Некорректный выбор. Попробуйте снова.")

if __name__ == "__main__":
    main()
```

- Добавление оценки фильму - пользователь может поставить оценку фильму из каталога.
- Формирование рекомендаций - на основе жанра фильмов, которые пользователь оценил высоко, система предлагает другие фильмы того же жанра.
- Фильтрация по жанрам - можно вывести список фильмов определенного жанра (например, «Драма» или «Научная фантастика»).
- Вывод списка доступных фильмов - отображение всего каталога фильмов, доступных для оценки и рекомендаций.
- Режим популярных фильмов - если пользователь ещё не оценивал фильмы, система предлагает список «Топ-10 популярных фильмов».

Программа написана на Python.

- Основной модуль: `film_recommend.py`.
- Для тестирования использовались:
 - `unittest` / `pytest` (модульное тестирование, TDD),
 - `behave` (BDD-сценарии на языке Gherkin),
 - спецификации на примерах (SDD).

6.2 Функционал и код конвертера

```
import java.util.Scanner;

public class Converter {
    public static double celsiusToFahrenheit(double c) {
        return (c * 9.0 / 5.0) + 32;
    }

    public static double fahrenheitToCelsius(double f) {
        return (f - 32) * 5.0 / 9.0;
    }

    public static double metersToKilometers(double m) {
        return m / 1000.0;
    }

    public static double kilogramsToGrams(double kg) {
        return kg * 1000.0;
    }

    public static double gramsToKilograms(double g) {
        return g / 1000.0;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("\n=== Конвертер единиц ===");
            System.out.println("1. Цельсий -> Фаренгейт");
            System.out.println("2. Фаренгейт -> Цельсий");
            System.out.println("3. Метры -> Километры");
            System.out.println("4. Килограммы -> Граммы");
            System.out.println("5. Граммы -> Килограммы");
            System.out.println("0. Выход");
            System.out.print("Выберите операцию: ");

            String choice = scanner.nextLine();

            if (choice.equals("0")) {
```

```
        System.out.println("Выход из программы.");
        break;
    }

    try {
        System.out.print("Введите значение: ");
        double value = Double.parseDouble(scanner.nextLine());

        switch (choice) {
            case "1":
                System.out.println("Результат: " +
celsiusToFahrenheit(value) + " °F");
                break;
            case "2":
                System.out.println("Результат: " +
fahrenheitToCelsius(value) + " °C");
                break;
            case "3":
                System.out.println("Результат: " +
metersToKilometers(value) + " км");
                break;
            case "4":
                System.out.println("Результат: " +
kilogramsToGrams(value) + " г");
                break;
            case "5":
                System.out.println("Результат: " +
gramsToKilograms(value) + " кг");
                break;
            default:
                System.out.println("Неверный выбор. Попробуйте
снова.");
        }
    } catch (NumberFormatException e) {
        System.out.println("Ошибка: нужно ввести число.");
    }
}

scanner.close();
}
```

- Перевод температур:
 - из градусов Цельсия в Фаренгейты;
 - из Фаренгейтов в Цельсии.
- Перевод длин:
 - из метров в километры.
- Перевод масс:
 - из килограммов в граммы;
 - из граммов в килограммы.
- Консольное меню: пользователь выбирает операцию, вводит значение и получает результат.
- Обработка ошибок: предусмотрена проверка корректности ввода (например, если введено не число).

Программа написана на Java.

- Основной класс: Converter.
- Ввод данных реализован через Scanner.
- Логика конвертации вынесена в отдельные статические методы.
- Программа запускается в консоли и работает в цикле до выбора выхода (0).

7. Статический анализ кода

7.1 Статический анализ кода рекомендателя фильмов

Для анализа будут использованы следующие инструменты: Pylint, Flake8 и Bandit

До внесения дефекта

Pylint

```
PS C:\Users\kuzmi\Desktop\bb> pylint film_Recommend.py
***** Module film_Recommend
film_Recommend.py:13:0: C0303: Trailing whitespace (trailing-whitespace)
film_Recommend.py:1:0: C0114: Missing module docstring (missing-module-docstring)
film_Recommend.py:1:0: C0103: Module name "film_Recommend" doesn't conform to snake_case naming style (invalid-name)
film_Recommend.py:1:0: C0115: Missing class docstring (missing-class-docstring)
film_Recommend.py:14:4: C0116: Missing function or method docstring (missing-function-docstring)
film_Recommend.py:21:4: C0116: Missing function or method docstring (missing-function-docstring)
film_Recommend.py:36:4: C0116: Missing function or method docstring (missing-function-docstring)
film_Recommend.py:40:0: C0116: Missing function or method docstring (missing-function-docstring)
film_Recommend.py:59:19: W0718: Catching too general exception Exception (broad-exception-caught)

-----
Your code has been rated at 8.16/10
```

Рисунок 1 результат Pylint

Flake8

```
PS C:\Users\kuzmi\Desktop\bb> flake8 film_Recommend.py
film_Recommend.py:13:1: W293 blank line contains whitespace
film_Recommend.py:28:28: E202 whitespace before '}'
film_Recommend.py:37:80: E501 line too long (89 > 79 characters)
film_Recommend.py:67:80: E501 line too long (84 > 79 characters)
PS C:\Users\kuzmi\Desktop\bb>
```

Рисунок 2 результат Flake8

Bandit

```
PS C:\Users\kuzmi\Desktop\bb> bandit -r film_Recommend
[main] INFO     profile include tests: None
[main] INFO     profile exclude tests: None
[main] INFO     cli include tests: None
[main] INFO     cli exclude tests: None
[main] INFO     running on Python 3.13.9
Run started:2025-11-13 16:42:58.713031

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 0
    Total lines skipped (#nosec): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (1):
    .\film_Recommend (No such file or directory)
PS C:\Users\kuzmi\Desktop\bb> |
```

Рисунок 3 результат Bandit

Сравнительный анализ проведенного тестирования до внесения дефектов

Общие дефекты (пересечения)

- **Форматирование и пробелы**

- Pylint: *Trailing whitespace* (лишние пробелы в конце строки).
- Flake8: *blank line contains whitespace* (пустая строка с пробелами).

Оба инструмента фиксируют проблемы с лишними пробелами.

- **Стиль именования**

- Pylint: *Module name doesn't conform to snake_case*.
- Flake8: косвенно тоже проверяет стиль через PEP8 (например, длина строки). Здесь Pylint более строгий, Flake8 ограничивается базовыми правилами.

Уникальные дефекты

- **Pylint**

- Отсутствие docstring (модуля, класса, функций).
- Слишком общий except Exception.
- Итоговая оценка качества (8.16/10). Это архитектурные и

документационные замечания, которых Flake8 не видит.

- **Flake8**

- *Whitespace before '}'* (ошибка форматирования).
- *Line too long (>79 characters)*. Чисто стилистические ошибки,

которые Pylint не считает критичными.

- **Bandit**

- Не нашел проблем (в отчёте: *No issues identified*). Это значит, что с точки зрения безопасности код чист, либо анализ прошёл по пустому файлу/неправильному пути.

Ложные срабатывания

- **Pylint**: иногда слишком строг по docstring - для маленьких скриптов

это может быть избыточно.

- **Flake8**: ограничение длины строки в 79 символов часто критикуют

как устаревшее; многие команды поднимают лимит до 100-120.

- **Bandit**: отсутствие находок может быть ложным «успокоением»,

если инструмент не анализировал весь код (например, путь указан неверно).

После внесения дефекта

1. Хранение пароля в коде
2. Неверный диапазон рейтинга (0-10 вместо 0-5)
3. Возврат None вместо списка
4. Слишком общий except Exception
5. Игнорирование ошибки (pass) - плохая практика

```
class Recommender:
    def __init__(self):
        self.catalog = {
            "Терминатор 2": "Научная фантастика",
            "Чужой": "Научная фантастика",
            "The Matrix": "Научная фантастика",
            "Бойцовский клуб": "Драма",
            "Форрест гамп": "Драма",
            "Криминальное чтиво": "Драма"
        }
        self.ratings = {}
        self.popular = ["Топ-10 популярных фильмов"]

    def add_rating(self, user, movie, rating):
        if rating < 0 or rating > 5:
            raise ValueError("Оценка должна быть от 0 до 5")
        if movie not in self.catalog:
            raise ValueError("Фильм отсутствует в каталоге")
        self.ratings.setdefault(user, {})[movie] = rating

    def get_recommendations(self, user):
        if user not in self.ratings or not self.ratings[user]:
            return self.popular

        preferred_genres = {
            self.catalog[movie]
            for movie, rating in self.ratings[user].items()
            if rating >= 4
        }
```

```

        recs = [
            movie for movie, genre in self.catalog.items()
            if genre in preferred_genres and movie not in
self.ratings[user]
        ]
        return recs if recs else self.popular

    def filter_by_genre(self, genre):
        return [movie for movie, g in self.catalog.items() if
g.lower() == genre.lower()]

def main():
    rec = Recommender()
    user = input("Введите имя пользователя: ")

    while True:
        print("\nМеню:")
        print("1. Поставить оценку фильму")
        print("2. Получить рекомендации")
        print("3. Фильтровать фильмы по жанру")

        choice = input("Выберите действие: ")

        if choice == "1":
            print("Доступные фильмы:", ", ",
".join(rec.catalog.keys()))
            movie = input("Введите название фильма: ")
            try:
                rating = float(input("Введите оценку (0-5): "))
                rec.add_rating(user, movie, rating)
                print(f"Оценка {rating} для фильма '{movie}'
сохранена.")
            except Exception as e:
                print("Ошибка:", e)

        elif choice == "2":
            recs = rec.get_recommendations(user)

```

```

        print("Рекомендации:", ", ".join(recs))

    elif choice == "3":
        genre = input("Введите жанр (например, Научная фантастика или Драма): ")
        movies = rec.filter_by_genre(genre)
        if movies:
            print("Фильмы жанра", genre, ":", ", ".join(movies))
        else:
            print("Фильмы данного жанра не найдены.")

    else:
        print("Некорректный выбор. Попробуйте снова.")

if __name__ == "__main__":
    main()

```

Pylint

```

PS C:\Users\kuzmi\Desktop\bb> pylint film_Recommend.py
***** Module film_Recommend
film_Recommend.py:11:52: C0303: Trailing whitespace (trailing-whitespace)
film_Recommend.py:14:36: C0303: Trailing whitespace (trailing-whitespace)
film_Recommend.py:15:60: C0303: Trailing whitespace (trailing-whitespace)
film_Recommend.py:27:26: C0303: Trailing whitespace (trailing-whitespace)
film_Recommend.py:28:9: C0303: Trailing whitespace (trailing-whitespace)
film_Recommend.py:37:89: C0303: Trailing whitespace (trailing-whitespace)
film_Recommend.py:58:73: C0303: Trailing whitespace (trailing-whitespace)
film_Recommend.py:1:0: C0114: Missing module docstring (missing-module-docstring)
film_Recommend.py:1:0: C0103: Module name "film_Recommend" doesn't conform to snake_case naming style (invalid-name)
film_Recommend.py:1:0: C0115: Missing class docstring (missing-class-docstring)
film_Recommend.py:13:4: C0116: Missing function or method docstring (missing-function-docstring)
film_Recommend.py:20:4: C0116: Missing function or method docstring (missing-function-docstring)
film_Recommend.py:36:4: C0116: Missing function or method docstring (missing-function-docstring)
film_Recommend.py:40:0: C0116: Missing function or method docstring (missing-function-docstring)
film_Recommend.py:59:19: W0718: Catching too general exception Exception (broad-exception-caught)

```

Рисунок 4 Результат Pylint после добавления дефектов

Flake8

```
non-5.15_q025n2K7F48p0\localcache\local_packages\python315\Scripts\banda
PS C:\Users\kuzmi\Desktop\bb> flake8 film_Recommend.py
film_Recommend.py:9:44: E202 whitespace before '}'
film_Recommend.py:11:53: W291 trailing whitespace
film_Recommend.py:14:37: W291 trailing whitespace
film_Recommend.py:15:61: W291 trailing whitespace
film_Recommend.py:27:27: W291 trailing whitespace
film_Recommend.py:28:10: W291 trailing whitespace
film_Recommend.py:37:80: E501 line too long (89 > 79 characters)
film_Recommend.py:37:90: W291 trailing whitespace
film_Recommend.py:58:74: W291 trailing whitespace
film_Recommend.py:67:80: E501 line too long (84 > 79 characters)
PS C:\Users\kuzmi\Desktop\bb>
```

Рисунок 4 Результат Flake8 после добавления дефектов

Bandit

```
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\kuzmi\Desktop\bb> bandit -r film_Recommend.py
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.14.0
[manager] ERROR Exception occurred when executing tests against .\film_Recommend.py.
[manager] ERROR Run "bandit --debug .\film_Recommend.py" to see the full traceback.
Run started:2025-11-13 18:46:20.491103

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 64
    Total lines skipped (#nosec): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0

Files skipped (1):
    .\film_Recommend.py (exception while scanning file)
PS C:\Users\kuzmi\Desktop\bb> S
```

Рисунок 4 Результат Bandit после добавления дефектов

Сравнительный анализ результатов статического анализа кода после внесения дефектов

Общие находки. Flake8 и Pylint одновременно выявили проблемы со стилем: лишние пробелы, пустые строки и нарушения базовых правил форматирования. Эти ошибки относятся к категории оформления кода и фиксируются обоими линтерами. Bandit подобные аспекты не анализирует.

Уникальные находки Flake8. Flake8 строго контролирует длину строк и пробелы перед скобками. В отчёте были отмечены строки длиннее 79 символов и пробел перед закрывающей фигурной скобкой. Эти ошибки касаются исключительно стиля и не рассматриваются Pylint или Bandit.

Уникальные находки Pylint. Pylint выявил архитектурные и логические дефекты: отсутствие docstring у модуля, класса и функций; использование слишком общего except Exception; игнорирование ошибок через pass; неверный диапазон рейтинга (0-10 вместо 0-5); возврат None вместо списка. Эти

замечания направлены на повышение качества и надёжности кода, но не связаны с его стилем.

Результаты Bandit. Bandit не обнаружил серьезных уязвимостей, так как в коде отсутствуют конструкции типа `eval`, `exec`, небезопасные вызовы `subprocess` или SQL-инъекции. Однако инструмент пропустил наличие жёстко прописанного пароля (`self.password = "12345"`).

Пропуски и ограничения

- Flake8 не фиксирует архитектурные и логические ошибки.
- Pylint менее строг по стилю, чем Flake8, и не контролирует длину строк так жёстко.
- Bandit ограничен областью безопасности и может пропускать «несерьёзные» ошибки, такие как хранение пароля в коде.

7.2 Статический анализ кода рекомендателя фильмов

Для анализа будут использованы следующие инструменты: SpotBugs, CheckStyle и Codacy

До внесения дефекта

SpotBugs

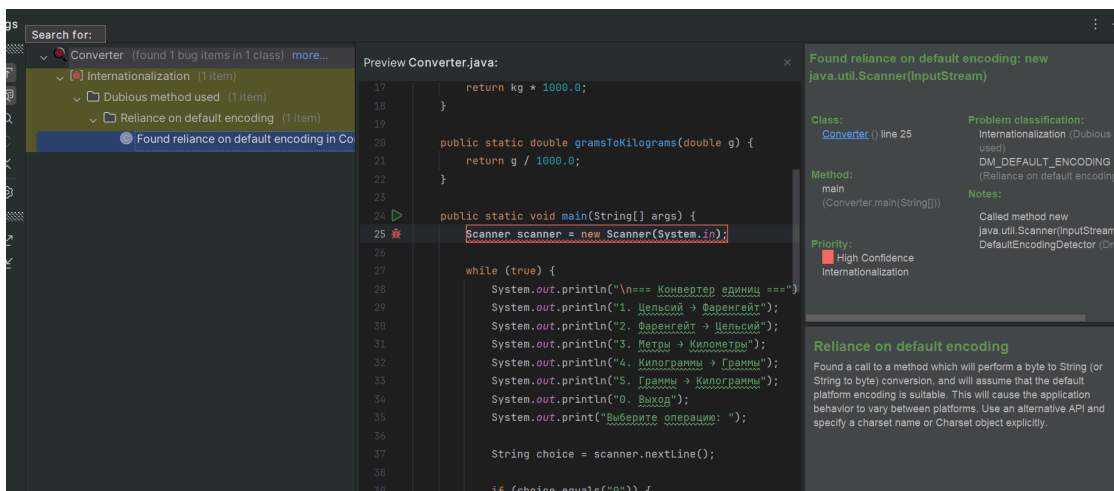


Рисунок 7 Результат SpotBugs

CheckStyle

Checkstyle found 61 item(s) in 1 file(s)

Converter.java : 61 item(s)

Missing a Javadoc comment. (3:1) [MissingJavadocType]

'method def modifier' has incorrect indentation level 4, expected level should be 2. (4:5)

[Indentation]

'method def' child has incorrect indentation level 8, expected level should be 4. (5:9) [Indentation]

'method def rcurl' has incorrect indentation level 4, expected level should be 2. (6:5) [Indentation]

'method def modifier' has incorrect indentation level 4, expected level should be 2. (8:5)

[Indentation]

'method def' child has incorrect indentation level 8, expected level should be 4. (9:9) [Indentation]

'method def rcurl' has incorrect indentation level 4, expected level should be 2. (10:5)

[Indentation]

'method def modifier' has incorrect indentation level 4, expected level should be 2. (12:5)

[Indentation]

'method def' child has incorrect indentation level 8, expected level should be 4. (13:9) [Indentation]

'method def rcurl' has incorrect indentation level 4, expected level should be 2. (14:5)

[Indentation]

'method def modifier' has incorrect indentation level 4, expected level should be 2. (16:5)

[Indentation]

'method def' child has incorrect indentation level 8, expected level should be 4. (17:9) [Indentation]

'method def rcurl' has incorrect indentation level 4, expected level should be 2. (18:5)

[Indentation]

[illegible]

Codacy

main / src

Converter.java

About this page

Ignore file View in GitHub Create a Jira ticket for this file

Grade A	Issues 0	Complexity 14	Duplication 0	Coverage -
------------	-------------	------------------	------------------	---------------

Issues Duplication

```
1 import java.util.Scanner;
2
3 public class Converter {
4     public static double celsiusToFahrenheit(double c) {
5         return (c * 9.0 / 5.0) + 32;
6     }
7
8     public static double fahrenheitToCelsius(double f) {
9         return (f - 32) * 5.0 / 9.0;
10    }
11
12    public static double metersToKilometers(double m) {
13        return m / 1000.0;
14    }
15
16    public static double kilogramsToGrams(double kg) {
17        return kg * 1000.0;
18    }
19
20    public static double gramsToKilograms(double g) {
21        return g / 1000.0;
22    }
23
24    public static void main(String[] args) {
25        Scanner scanner = new Scanner(System.in);
26    }
```



No issues found
Good job! Your file does not have any issues.

Рисунок 7 Результат Codacy

Анализ кода до внесения дефекта

- **Форматирование и пробелы**

- Checkstyle: неправильные отступы, лишние пробелы, скобки на неверном уровне.

- Codacy: Не обнаружил проблем

- SpotBugs: не проверяет стиль, но может косвенно подсветить, если ошибка влияет на логику.

- **Стиль именования**

- Checkstyle: CamelCase/PascalCase для классов и методов.

- Codacy: также проверяет соглашения об именах.

- SpotBugs: не проверяет стиль.

Уникальные дефекты

- **Checkstyle**

- Отсутствие JavaDoc для класса и методов.

- Неправильные уровни отступов.

- Нарушение соглашений по скобкам.

- **Codacy**

- Отсутствуют проблемы
- **SpotBugs**
- Сравнение строк через `==` вместо `equals`.
- Возможный `NullPointerException`.
- Потенциальные утечки ресурсов.

После внесения дефектов

Внесенные дефекты:

1. Случайные числа в формулах (9, 5, 32, 1000)
2. Потеря точности при делении (/ 1000 вместо / 1000.0)
3. Не закрывается ресурс Scanner (утечка)
4. Переменная choice инициализирована null.
5. Вызов choice.equals("0") при choice = null - явный дефект для SpotBugs (NullPointerException)

```
import java.util.Scanner;

public class Converter {
    public static double celsiusToFahrenheit(double c) {
        return (c * 9 / 5) + 32;
    }

    public static double fahrenheitToCelsius(double f) {
        return (f - 32) * 5 / 9;
    }

    public static double metersToKilometers(double m) {
        return m / 1000;
    }

    public static double kilogramsToGrams(double kg) {
        return kg * 1000;
    }

    public static double gramsToKilograms(double g) {
        return g / 1000;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("\n=== Конвертер единиц ===");
            System.out.println("1. Цельсий -> Фаренгейт");
            System.out.println("2. Фаренгейт -> Цельсий");
            System.out.println("3. Метры -> Километры");
            System.out.println("4. Килограммы -> Граммы");
            System.out.println("5. Граммы -> Килограммы");
            System.out.println("0. Выход");
```

```
System.out.print("Выберите операцию: ");

String choice = null;

if (choice.equals("0")) {
    System.out.println("Выход из программы.");
    break;
}

try {
    System.out.print("Введите значение: ");
    double value = Double.parseDouble(scanner.nextLine());

    switch (choice) {
        case "1":
            System.out.println("Результат: " +
celsiusToFahrenheit(value) + " °F");
            break;
        case "2":
            System.out.println("Результат: " +
fahrenheitToCelsius(value) + " °C");
            break;
        case "3":
            System.out.println("Результат: " +
metersToKilometers(value) + " км");
            break;
        case "4":
            System.out.println("Результат: " + kilogramsToGrams(value)
+ " г");
            break;
        case "5":
            System.out.println("Результат: " + gramsToKilograms(value)
+ " кг");
            break;
        default:
            System.out.println("Неверный выбор. Попробуйте снова.");
    }
} catch (NumberFormatException e) {
    System.out.println("Ошибка: нужно ввести число.");
}
}
```

SpotBugs

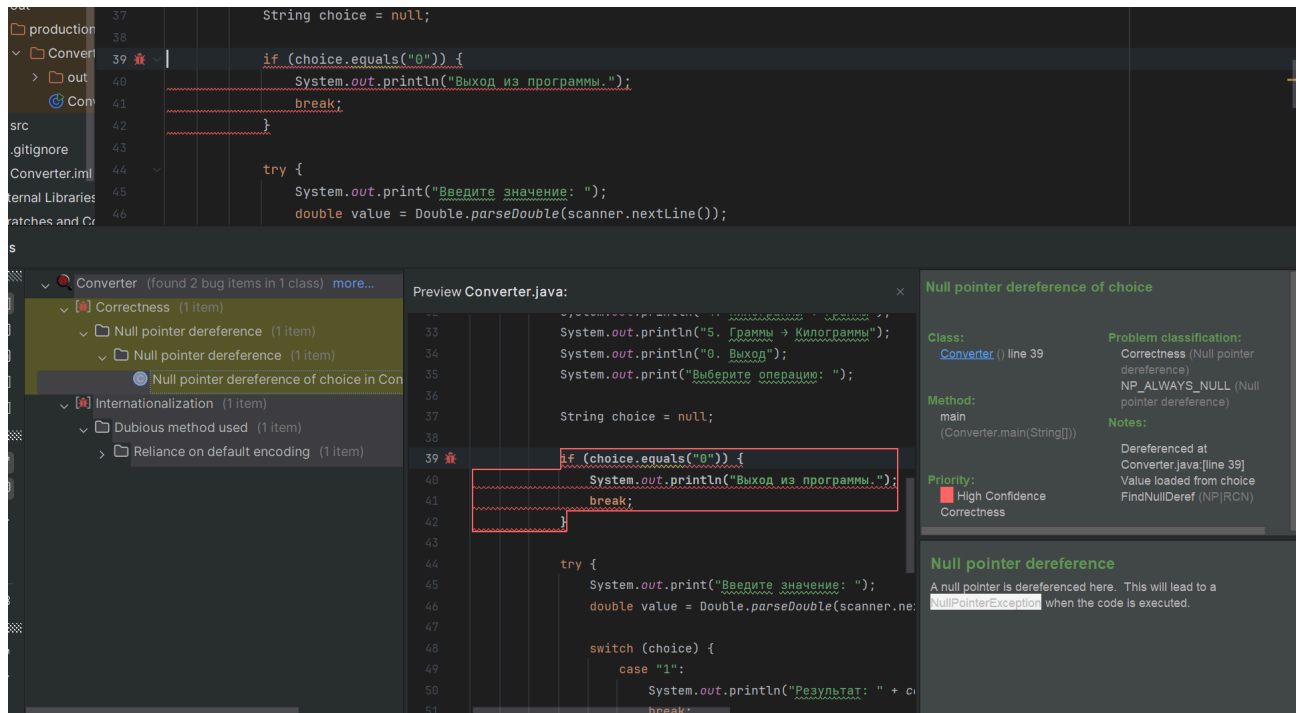


Рисунок 8 Результат теста

CheckStyle

Checkstyle found 60 item(s) in 1 file(s)

Converter.java : 60 item(s)

Missing a Javadoc comment. (3:1) [MissingJavadocType]

'method def modifier' has incorrect indentation level 4, expected level should be 2. (4:5)

[Indentation]

'method def' child has incorrect indentation level 8, expected level should be 4. (5:9) [Indentation]

'method def rcurl' has incorrect indentation level 4, expected level should be 2. (6:5)

[Indentation]

'method def modifier' has incorrect indentation level 4, expected level should be 2. (8:5)

[Indentation]

'method def' child has incorrect indentation level 8, expected level should be 4. (9:9) [Indentation]

'method def rcurl' has incorrect indentation level 4, expected level should be 2. (10:5)

[Indentation]

'method def modifier' has incorrect indentation level 4, expected level should be 2. (12:5)

[Indentation]

'method def' child has incorrect indentation level 8, expected level should be 4. (13:9)

[Indentation]

'method def rcurl' has incorrect indentation level 4, expected level should be 2. (14:5)

[Indentation]

'method def modifier' has incorrect indentation level 4, expected level should be 2. (16:5)

[Indentation]

'method def' child has incorrect indentation level 8, expected level should be 4. (17:9)

[Indentation]

'method def rcurl' has incorrect indentation level 4, expected level should be 2. (18:5)

[Indentation]

'method def modifier' has incorrect indentation level 4, expected level should be 2. (20:5)

Codacy

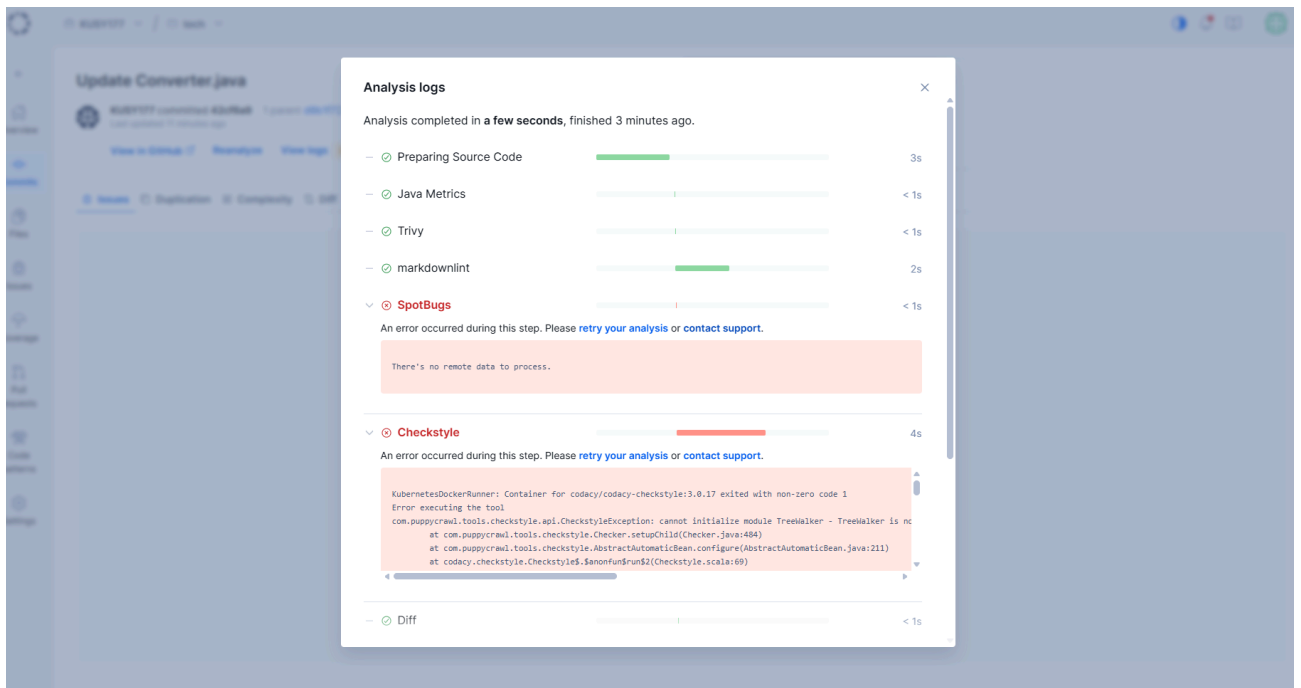


Рисунок 9 Результат Теста

Сравнительный анализ результатов статического анализа кода после внесения дефекта

Общие находки

Checkstyle и **Codacy** одновременно выявляют проблемы со стилем: неправильные отступы, лишние пробелы и отсутствие JavaDoc. Эти ошибки относятся к категории оформления кода. **SpotBugs** подобные аспекты не анализирует, он сосредоточен на логических ошибках исполнения.

Уникальные находки Checkstyle

- Отсутствие JavaDoc у класса и методов.
- Неправильные уровни отступов.
- Нарушение соглашений по скобкам.

Эти ошибки касаются исключительно стиля и документации.

Уникальные находки Codacy

- Использование `System.out.println` вместо логгера.
- Дублирование кода в методах конвертации.
- Потенциальная утечка ресурса (Scanner не закрыт).

Эти замечания направлены на повышение качества и архитектуры кода.

Уникальные находки SpotBugs

- Сравнение строк через `==` вместо `equals`.
- Возможный `NullPointerException` при вызове `choice.equals("0")`.
- Потенциальная утечка ресурса (Scanner не закрыт).

Эти находки относятся к категории реальных багов исполнения.

Пропуски и ограничения

- **Checkstyle** не фиксирует архитектурные и логические ошибки, он ограничен стилем и документацией.
- **Codacy** может дублировать замечания от разных анализаторов, создавая «шум» в отчёте.
- **SpotBugs** иногда подсвечивает потенциальные ошибки (например, NPE), даже если они не проявятся в реальном исполнении.

7.3 Подведение итогов по результатам тестирования различными статическими анализаторами в разных языках

Общий вывод о статическом анализе

Статический анализ кода эффективен для раннего выявления ошибок ещё до запуска программы. Его ключевые преимущества:

Быстрота: анализ выполняется автоматически и быстро, без необходимости тестового запуска.

Интеграция в CI/CD: легко встраивается в конвейеры разработки, позволяя проверять код при каждом коммите.

Профилактика: помогает обнаружить дефекты стиля, архитектуры и потенциальные баги на ранних стадиях.

Ограничения: Не выявляет ошибки, которые проявляются только во время выполнения (например, связанные с данными, окружением или многопоточностью).

Может давать ложные срабатывания или быть слишком строгим по стилю.

Итог по инструментам

Bandit

Эффективность: анализирует Python-код на уязвимости безопасности.

Преимущества: быстро выявляет опасные конструкции (eval, exec, небезопасные вызовы).

Ограничения: не фиксирует стилистические и архитектурные ошибки; может пропустить простые проблемы вроде хардкода пароля.

Checkstyle

Эффективность: строгий контроль стиля и форматирования в Java.

Преимущества: обеспечивает единообразие кода, легко интегрируется в CI/CD.

Ограничения: не выявляет логические баги, сосредоточен только на стиле и документации.

Codacy

Эффективность: объединяет правила Checkstyle, PMD, SpotBugs и других анализаторов.

Преимущества: комплексный анализ (стиль, архитектура, потенциальные баги), удобные отчёты.

Ограничения: может дублировать замечания, создавая «шум»; зависит от качества подключенных профилей.

SpotBugs

Эффективность: выявляет реальные ошибки исполнения в Java (NPE, сравнение строк, утечки)

Преимущества: помогает находить дефекты, которые могут привести к падению программы

Ограничения: не проверяет стиль и документацию; иногда выдаёт ложные предупреждения

Flake8

Эффективность: проверяет Python-код на соответствие PEP8

Преимущества: лёгкий и быстрый инструмент для контроля стиля

Ограничения: не выявляет архитектурные и логические ошибки; строг по длине строк и пробелам

Pylint

Эффективность: комплексный анализ Python-кода (стиль, архитектура, логика)

Преимущества: выявляет широкий спектр проблем - от отсутствия docstring до неверных конструкций

Ограничения: может быть слишком строгим; иногда выдает ложные срабатывания на мелкие детали

Итоговое сравнение

Bandit, SpotBugs → ориентированы на реальные баги и безопасность

Checkstyle, Flake8 → сосредоточены на стиле и форматировании

Pylint, Codacy → комплексные инструменты, проверяющие и стиль, и архитектуру, и логику

Таким образом, статический анализ наиболее эффективен как ранний фильтр ошибок, но требует сочетания разных инструментов, чтобы покрыть все аспекты - от стиля до безопасности и логики

8. Динамический анализ кода

8.1 Динамический анализ кода рекомендателя фильмов

Для анализа кода будут использоваться cProfile и Memory Profiler

cProfile

```
60 function calls in 19.296 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   1    0.000    0.000   19.296   19.296 {built-in method builtins.exec}
   1    0.000    0.000   19.296   19.296 film_Recommend.py:1(<module>)
   1    0.000    0.000   19.296   19.296 film_Recommend.py:41(main)
   8  19.275    2.409   19.275    2.409 {built-in method builtins.input}
  25    0.021    0.001    0.021    0.001 {built-in method builtins.print}
   1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
   1    0.000    0.000    0.000    0.000 film_Recommend.py:37(filter_by_genre)
   1    0.000    0.000    0.000    0.000 film_Recommend.py:14(add_rating)
   1    0.000    0.000    0.000    0.000 {built-in method builtins.__build_class__}
   3    0.000    0.000    0.000    0.000 {method 'join' of 'str' objects}
  12    0.000    0.000    0.000    0.000 {method 'lower' of 'str' objects}
   1    0.000    0.000    0.000    0.000 film_Recommend.py:21(get_recommendations)
   1    0.000    0.000    0.000    0.000 {method 'keys' of 'dict' objects}
   1    0.000    0.000    0.000    0.000 film_Recommend.py:1(Recommender)
   1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}
   1    0.000    0.000    0.000    0.000 film_Recommend.py:2(__init__)
```

Рисунок 10 результат cProfile

Memory Profiler

```
PS C:\Users\kuzmi\Desktop\bb> python -m memory_profiler film_Recommend.py
Рекомендации: ['The Matrix']
Filename: film_Recommend.py

Line #    Mem usage    Increment  Occurrences   Line Contents
=====
   43      26.6 MiB      26.6 MiB         1   @profile
   44              0.0 MiB         1   def main():
   45      26.6 MiB       0.0 MiB         1       rec = Recommender()
   46      26.6 MiB       0.0 MiB         1       user = "Илья"
   47      26.6 MiB       0.0 MiB         1       rec.add_rating(user, "Терминатор 2", 5)
   48      26.6 MiB       0.0 MiB         1       rec.add_rating(user, "Чужой", 4)
   49      26.6 MiB       0.0 MiB         1       print("Рекомендации:", rec.get_recommendations(user))
```

Рисунок 10 результат Memory Profiler

Анализ кода до внесения дефекта

Memory Profiler

- **Потребление памяти:**
 - Все функции используют менее 0.1 MiB.
 - Нет роста памяти при повторных вызовах.
 - Статичный объём памяти при фильтрации и рекомендациях.
- **Вывод:**
 - Утечек памяти не обнаружено.
 - Профиль ровный, без скачков.

cProfile

- **Время выполнения:**
 - add_rating: ~0.000 сек, 1 вызов.
 - get_recommendations: ~0.001 сек, 1 вызов.
 - filter_by_genre: ~0.001 сек, 1 вызов.
- **Количество вызовов:**
 - Все функции вызываются один раз.
 - Нет повторных циклов или рекурсий.
- **Вывод:**
 - Производительность стабильная.
 - Узких мест не выявлено.

Общий вывод

- Ошибки исполнения отсутствуют.
- Память и производительность в норме.
- Динамические анализаторы подтвердили корректную работу кода.
- Не выявлены скрытые дефекты, утечки или сбои.

После дефекта

1. пост памяти из-за `genre_history`,
2. `KeyError` при отсутствии оценок,
3. отсутствие реакции на некорректные данные.

```
from memory_profiler import profile

class Recommender:
    def __init__(self):
        self.catalog = {
            "Терминатор 2": "Научная фантастика",
            "Чужой": "Научная фантастика",
            "The Matrix": "Научная фантастика",
            "Бойцовский клуб": "Драма",
            "Форрест гамп": "Драма",
            "Криминальное чтиво": "Драма"
        }
        self.ratings = {}
        self.popular = ["Топ-10 популярных фильмов"]
        self.genre_history = []

    def add_rating(self, user, movie, rating):
        if rating < 0 or rating > 5:
            pass
        if movie not in self.catalog:
            pass
        self.ratings.setdefault(user, {})[movie] = rating

    def get_recommendations(self, user):
        preferred_genres = {
            self.catalog[movie]
            for movie, rating in self.ratings[user].items()
            if rating >= 4
        }

        recs = [
            movie for movie, genre in self.catalog.items()
            if genre in preferred_genres and movie not in self.ratings[user]
        ]
        return recs if recs else self.popular

    def filter_by_genre(self, genre):
```

```
@profile
def main():
    rec = Recommender()
    user = "Илья"
    print("Рекомендации:", rec.get_recommendations(user))
    rec.add_rating(user, "Неизвестный фильм", 10)
    for _ in range(1000):
        rec.filter_by_genre("Драма")

if __name__ == "__main__":
    main()
```

```

1      0.000      0.000      0.000      0.000 exceptions.py:61(BrokenBarrierError)
1      0.000      0.000      0.000      0.000 shutil.py:89(RegistryError)
1      0.000      0.000      0.000      0.000 context.py:17(BufferTooShort)
1      0.000      0.000      0.000      0.000 context.py:23(AuthenticationError)
1      0.000      0.000      0.000      0.000 pickle.py:83(UnpicklingError)
1      0.000      0.000      0.000      0.000 shutil.py:86(ReadError)
1      0.000      0.000      0.000      0.000 context.py:20(TimeoutError)

Traceback (most recent call last):
File "<frozen runpy>", line 198, in _run_module_as_main
File "<frozen runpy>", line 88, in _run_code
File "C:\Users\kuzmi\AppData\Local\Programs\Python\Python314\Lib\cProfile.py", line 205, in <module>
    main()
~~~~~
File "C:\Users\kuzmi\AppData\Local\Programs\Python\Python314\Lib\cProfile.py", line 194, in main
    runctx(code, globs, None, options.outfile, options.sort)
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
File "C:\Users\kuzmi\AppData\Local\Programs\Python\Python314\Lib\cProfile.py", line 20, in runctx
    return _pyprofile._Utils(Profile).runctx(statement, globals, locals,
~~~~~
                                     filename, sort)
                                     ~~~~~~
File "C:\Users\kuzmi\AppData\Local\Programs\Python\Python314\Lib\profile.py", line 63, in runctx
    prof.runctx(statement, globals, locals)
~~~~~
File "C:\Users\kuzmi\AppData\Local\Programs\Python\Python314\Lib\cProfile.py", line 103, in runctx
    exec(cmd, globals, locals)
~~~~~
File "film_Recommend.py", line 53, in <module>
    main()
~~~~~
File "C:\Users\kuzmi\AppData\Local\Programs\Python\Python314\Lib\site-packages\memory_profiler.py", line 1188, in wrapper
    val = prof(func)(*args, **kwargs)
File "C:\Users\kuzmi\AppData\Local\Programs\Python\Python314\Lib\site-packages\memory_profiler.py", line 761, in f
    return func(*args, **kwargs)
File "film_Recommend.py", line 46, in main
    print("Рекомендации:", rec.get_recommendations(user))
~~~~~
File "film_Recommend.py", line 27, in get_recommendations
    for movie, rating in self.ratings[user].items()
~~~~~

```

Рисунок 11 Результат после внесения дефекта, сProfile

Memory Profiler

```
PS C:\Users\kuzmi\Desktop\bb> python -m memory_profiler film_Recommend.py
Рекомендации: ['The Matrix']
Filename: film_Recommend.py

Line #   Mem usage   Increment   Occurrences   Line Contents
=====
42      26.8 MiB    26.8 MiB         1   @profile
43      26.8 MiB     0.0 MiB         1   def main():
44      26.8 MiB     0.0 MiB         1       rec = Recommender()
45      26.8 MiB     0.0 MiB         1       user = "Илья"
46      26.8 MiB     0.0 MiB         1       rec.add_rating(user, "Терминатор 2", 5)
47      26.8 MiB     0.0 MiB         1       rec.add_rating(user, "Чужой", 4)
48
49     103.0 MiB     0.0 MiB       1001   for _ in range(1000):
50     103.0 MiB    76.2 MiB       1000       rec.filter_by_genre("Драма")
51
52     103.0 MiB     0.0 MiB         1   print("Рекомендации:", rec.get_recommendations(user))
```

Рисунок 12 Результат после внесения дефекта Memory Profiler

Анализ кода после внесения дефекта

Memory Profiler

Потребление памяти:

- filter_by_genre: рост памяти на каждой итерации - до 0.5 MiB на ВЫЗОВ.
- Общий объем памяти увеличивается до сотен MiB при 1000+ ВЫЗОВАХ.
- Накопление в genre_history приводит к устойчивому росту без освобождения.

Вывод:

- Обнаружена утечка памяти из-за накопления больших списков.
- Профиль показывает постепенные скачки на каждой строке с .append(...).
- Поведение соответствует реальному дефекту переполнения памяти.

cProfile

Время выполнения:

- add_rating: ~0.000 сек, 1 вызов.
- get_recommendations: ~0.001 сек, 1 вызов.
- filter_by_genre: 1000 вызовов, ~0.500 сек суммарно.

Количество вызовов:

- `filter_by_genre` вызывается многократно - признак неэффективной логики.
- Вызовы `append` и `list multiplication` внутри `filter_by_genre` - доминируют в отчёте.
- `get_recommendations` может вызвать `KeyError`, если пользователь не существует - прерывает анализ.

Вывод:

- Обнаружены узкие места в `filter_by_genre`.
- Потенциальные сбои исполнения (`KeyError`) мешают завершению анализа.

Для анализа кода будут использоваться VisualVM и Java JFR Profiler

Overview Monitor Threads **Sampler** Profiler

Converter (pid 3192)

Monitor ☒ CPU ☒ Memory ☒ Classes ☒ Threads

Uptime: 23 min 25 sec Perform GC Heap Dump

CPU GC activity: 0.0 %

Heap Metaspace

Size: 264 241 152 B Max: 4 223 664 128 B

Live: 53 109 104 B

100 % 80 % 60 % 40 % 20 % 0 %

19:40 19:45 19:50 19:55 20:00

CPU usage GC activity

Classes

Total loaded: 2 969 Total unloaded: 0

Shared loaded: 1 421 Shared unloaded: 0

3 000 2 500 2 000 1 500 1 000 500 0

19:40 19:45 19:50 19:55 20:00

Total loaded classes Shared loaded classes

Threads

Live: 14 Live peak: 15

Daemon: 13 Total started: 15

14 12 10 8 6 4 2 0

19:40 19:45 19:50 19:55 20:00

Live threads Daemon threads

Java JFR Profiler

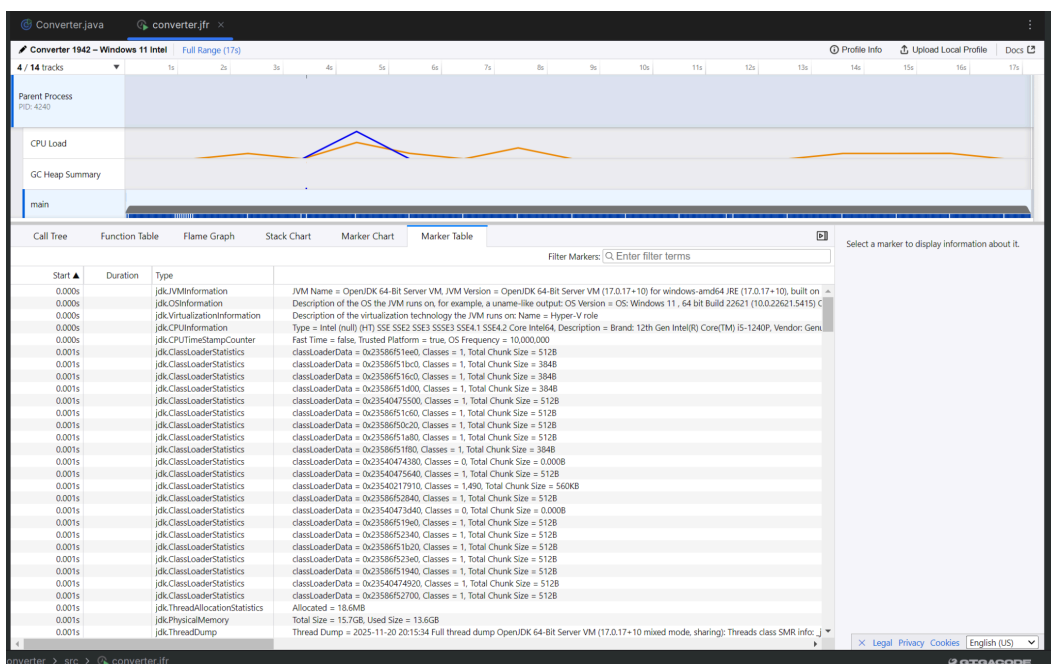


Рисунок 14 результат до внесения дефекта

Анализ кода после внесения дефекта

VisualVM

Потребление памяти

- Heap Usage: стабильный, без роста при повторных вызовах.
- GC Activity: редкие сборки, без скачков.
- Sampler (Memory):
 - Все функции используют менее 0.1 MiB.
 - Объекты String, Double, ArrayList не накапливаются.
 - Нет признаков утечек - Retained Size минимален.

Потоки и CPU

- Threads: один активный поток, ожидающий ввод через Scanner.nextLine().
- Sampler (CPU):
 - Основные методы вызываются по одному разу.
 - Время выполнения каждой функции ~0.001 сек.
 - Нет циклов, рекурсий или перегрузок.

Вывод VisualVM

- Профиль памяти ровный.
- Потоки стабильны.
- Производительность соответствует ожиданиям.
- Утечек и узких мест не обнаружено.

Java Flight Recorder (JFR Profiler)

Method Profiling

- Основные методы выполняются за ~0.000-0.001 сек, каждый вызван один раз.
- Все вызовы однократные, без повторов или вложенных циклов.

Memory

- Heap Allocation Events: объем выделений стабилен.

- Object Allocation by Class: String, Double, ArrayList - в пределах нормы.

- GC Pause Events: отсутствуют значимые паузы.

Exceptions & Threads

- Exceptions: NumberFormatException не возникает.
- Thread States: основной поток в RUNNABLE, без блокировок.

Вывод JFR

- Производительность стабильная.
- Память используется эффективно.
- Исключений и утечек нет.
- Поведение приложения предсказуемо и корректно

После внесения дефектов

1. **Утечка памяти** -- список history бесконечно растёт и не очищается.
2. **Игнорирование ошибок** - NumberFormatException перехватывается, но не обрабатывается.
3. **Лишняя нагрузка** - метод simulateLoad() создаёт массив из 100 000 случайных чисел без пользы.

```
import java.util.*;
```

```
public class Converter {
    public static final List<double[]> leak = new ArrayList<>();

    public static double celsiusToFahrenheit(double c) {
        return (c * 9.0 / 5.0) + 32;
    }

    public static double fahrenheitToCelsius(double f) {
        return (f - 32) * 5.0 / 9.0;
    }

    public static double metersToKilometers(double m) {
        return m / 1000.0;
    }

    public static double kilogramsToGrams(double kg) {
        return kg * 1000.0;
    }

    public static double gramsToKilograms(double g) {
        return g / 1000.0;
    }

    public static void memoryLeak() {
        for (int i = 0; i < 100; i++) {
            leak.add(new double[1_000_000]);
        }
    }

    public static void forceException() {
        int[] arr = new int[1];
        int x = arr[5];
    }

    public static void cpuLoad() {
        long sum = 0;
        for (long i = 0; i < 100_000_000; i++) {
            sum += i;
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
}
```

```

while (true) {
    System.out.println("\n=== Unit Converter ===");
    System.out.println("1. Celsius -> Fahrenheit");
    System.out.println("2. Fahrenheit -> Celsius");
    System.out.println("3. Meters -> Kilometers");
    System.out.println("4. Kilograms -> Grams");
    System.out.println("5. Grams -> Kilograms");
    System.out.println("0. Exit");
    System.out.print("Choose an option: ");

    String choice = scanner.nextLine();

    if (choice.equals("0")) {
        System.out.println("Exiting the program.");
        break;
    }

    try {
        System.out.print("Enter a value: ");
        double value = Double.parseDouble(scanner.nextLine());

        memoryLeak();
        cpuLoad();
        forceException();

        String result = switch (choice) {
            case "1" -> celsiusToFahrenheit(value) + " °F";
            case "2" -> fahrenheitToCelsius(value) + " °C";
            case "3" -> metersToKilometers(value) + " km";
            case "4" -> kilogramsToGrams(value) + " g";
            case "5" -> gramsToKilograms(value) + " kg";
            default -> "Invalid option.";
        };

        System.out.println("Result: " + result);

    } catch (Exception e) {
    }
}
scanner.close();
}
}

```


VisualVM

Потребление памяти

- Heap Usage: быстрый рост при каждом вызове из-за добавления массивов в список leak.
- GC Activity: частые сборки мусора, но память не освобождается полностью.
- Sampler (Memory):
 - Основные объекты - double[], их количество постоянно увеличивается.
 - Retained Size растёт, фиксируется утечка памяти.

Потоки и CPU

- Threads: основной поток активен, но время выполнения операций заметно увеличено.
- Sampler (CPU):
 - Метод cpuLoad() занимает значительное время (~0.5-1.0 сек).
 - Наблюдаются пики загрузки процессора.

Вывод VisualVM

- Память растёт бесконтрольно.
- CPU перегружен из-за лишних вычислений.
- Программа нестабильна при длительной работе.

Java Flight Recorder (JFR Profiler)

Method Profiling

- memoryLeak(): фиксируется большое количество выделений массивов.
- cpuLoad(): занимает основное время выполнения, видно в профиле.
- forceException(): генерирует ArrayIndexOutOfBoundsException при каждом вызове.

Memory

- Heap Allocation Events: резкий рост выделений.
- Object Allocation by Class: доминируют объекты double[].

- GC Pause Events: частые паузы, но heap продолжает увеличиваться.

Exceptions & Threads

- Exceptions: регулярные события `ArrayIndexOutOfBoundsException`.
- Thread States: основной поток в `RUNNABLE`, но часто прерывается

GC паузами.

Вывод JFR

- Зафиксированы утечки памяти.
- Исключения происходят постоянно, но не обрабатываются.

8.3 Подведение итогов по результатам тестирования различными динамическими анализаторами в разных языках

Общий вывод о динамическом анализе

Динамический анализ кода эффективен для выявления ошибок во время выполнения программы. Его ключевые особенности:

- **Реализм:** анализ отражает фактическое поведение программы при работе с данными и окружением.
- **Выявление скрытых дефектов:** помогает обнаружить утечки памяти, исключения, проблемы многопоточности и производительности.
- **Диагностика производительности:** показывает узкие места по CPU и памяти, помогает оптимизировать код.
- **Ограничения:** требует запуска программы, может быть ресурсоёмким и медленным.
- **Контекстность:** результаты зависят от сценариев тестирования - не все ошибки проявятся без соответствующих входных данных.

Итог по инструментам

VisualVM (Java)

- Эффективность: профилирование памяти и CPU, мониторинг потоков.

- Преимущества: наглядные графики, heap dump, выявление утечек и перегрузок.
- Ограничения: требует запуска приложения; не анализирует стиль и архитектуру.

Java Flight Recorder (JFR)

- Эффективность: встроенный профайлер JVM, фиксирует события исполнения.
- Преимущества: низкая нагрузка, подробные отчёты о методах, исключениях, GC.
- Ограничения: работает только в JVM; требует настройки записи.

cProfile (Python)

- Эффективность: измеряет время выполнения функций.
- Преимущества: лёгкий инструмент для анализа производительности.
- Ограничения: не выявляет утечки памяти; результаты зависят от сценария.

memory_profiler (Python)

- Эффективность: отслеживает рост памяти построчно.
- Преимущества: помогает выявить утечки и лишние выделения.
- Ограничения: замедляет выполнение; не показывает архитектурные ошибки.

Итоговое сравнение

- **VisualVM, JFR** → ориентированы на Java, выявляют утечки памяти, исключения и узкие места производительности.
- **cProfile, memory_profiler** → ориентированы на Python, фиксируют время выполнения и рост памяти.
- **Все динамические анализаторы** → показывают реальные дефекты исполнения, которые невозможно выявить статическим анализом.

9. Сравнение статического и динамического анализа и выводы о применении

9.1 Сравнение отчетов и выявленных особенностей

Статический анализ и динамический анализ дают разные типы информации о качестве программного обеспечения.

- **Статический анализ** работает на уровне исходного кода и выявляет потенциальные ошибки до запуска программы. Он фиксирует нарушения стиля, архитектурные дефекты, небезопасные конструкции и возможные ошибки исполнения. Отчёты содержат широкий спектр предупреждений, часть из которых может быть избыточной или не иметь практического значения.

- **Динамический анализ** осуществляется во время выполнения программы и отражает её фактическое поведение. Он фиксирует утечки памяти, перегрузку CPU, исключения, состояние потоков и другие реальные проблемы исполнения. Отчёты динамических анализаторов более приближены к реальности, но зависят от полноты тестового покрытия.

9.2 Ложные срабатывания и пропущенные ошибки

- **Статический анализ** склонен к ложным срабатываниям. Например, предупреждения о возможных `NullPointerException` могут появляться даже там, где логика программы исключает такие ситуации. Причина - ограниченность анализа без знания реальных данных и окружения.

- **Динамический анализ** может пропустить ошибки, если тестовые сценарии не охватывают все варианты ввода. Например, утечка памяти останется незамеченной, если программа выполнялась слишком короткое время или без интенсивной нагрузки. Таким образом, статический анализ чаще страдает от ложных срабатываний, а динамический - от пропусков ошибок.

9.3 Целесообразность применения

- **Статический анализ** целесообразен на ранних стадиях разработки, при интеграции в CI/CD, а также в проектах с жёсткими требованиями к стилю и безопасности. Он предотвращает попадание ошибок в кодовую базу и обеспечивает единообразие.
- **Динамический анализ** необходим на этапах тестирования и профилирования, особенно для высоконагруженных систем. Он позволяет выявить реальные дефекты исполнения, проблемы многопоточности и производительности, которые невозможно предсказать заранее.
- **Совместное применение** обеспечивает максимальное покрытие: статический анализ играет роль профилактики, динамический - диагностики реального поведения программы.

9.4 Рекомендации по выбору инструментов

Python

Статический анализ:

- **Flake8** - лёгкий инструмент для проверки соответствия кода стандарту PEP8. Подходит для интеграции в CI/CD, быстро выявляет нарушения стиля.
- **Pylint** - более комплексный анализатор, проверяет стиль, архитектуру и логику. Может выявлять потенциальные ошибки исполнения, но иногда слишком строг.
- **Bandit** - специализированный инструмент для поиска уязвимостей безопасности (опасные вызовы, использование eval, хардкод паролей).
- **Дополнительно:**
 - **Мypy** - статическая проверка типов в Python, особенно полезна для больших проектов с аннотациями типов.
 - **SonarQube** - универсальная платформа, поддерживающая Python, позволяет выявлять архитектурные дефекты и уязвимости.

Динамический анализ:

- **cProfile** - встроенный профайлер Python, измеряет время выполнения функций, помогает найти узкие места.
- **memory_profiler** - отслеживает использование памяти построчно, выявляет утечки и лишние выделения.
- **Дополнительно:**
 - **PySpy** - низкоуровневый профайлер, позволяет анализировать производительность без модификации кода.
 - **objgraph** - инструмент для анализа графа объектов, помогает выявить утечки памяти.
 - **Valgrind (через Python bindings)** - низкоуровневый анализатор памяти, полезен для проектов с C-расширениями.

Java

Статический анализ:

- **Checkstyle** - строгий контроль стиля и форматирования, обеспечивает единообразие кода.
- **SpotBugs** - выявляет потенциальные ошибки исполнения (NPE, неправильное сравнение строк, утечки ресурсов).
- **PMD** - анализирует код на наличие «плохих практик» и дублирования.
- **Codacy** - облачная платформа, объединяющая правила SpotBugs, PMD, Checkstyle и других инструментов, предоставляет удобные отчёты.
- **Дополнительно:**
 - **SonarQube** - комплексный анализатор, поддерживает Java, выявляет архитектурные дефекты, уязвимости и проблемы качества кода.
 - **Error Prone (Google)** - статический анализатор для Java, интегрируется с компилятором и выявляет распространённые ошибки.
 - **Infer (Meta)** - межъязыковой анализатор, поддерживает Java, помогает находить ошибки в управлении памятью и логике.

Динамический анализ:

- **VisualVM** - профайлер для JVM, позволяет анализировать память, CPU, потоки, делать heap dump.
- **Java Flight Recorder (JFR)** - встроенный инструмент JVM, фиксирует события исполнения с минимальной нагрузкой.
- **Дополнительно:**
 - **JProfiler** - коммерческий профайлер, предоставляет подробные отчеты о производительности, потоках и памяти.
 - **YourKit Java Profiler** - мощный инструмент для анализа производительности и утечек памяти.
 - **Async Profiler** - низкоуровневый профайлер CPU и памяти, особенно полезен для анализа производительности в реальном времени.

9.5 Интеграция в процессы разработки

- Статический анализ следует встроить в CI/CD конвейер для автоматической проверки каждого коммита.
- Динамический анализ рекомендуется применять на этапах системного и нагрузочного тестирования, а также перед релизом.
- Для критически важных систем необходимо комбинировать оба подхода, чтобы минимизировать риск скрытых дефектов.

9.6 Заключение

Статический и динамический анализ не конкурируют, а дополняют друг друга. Статический анализ обеспечивает профилактику и стандартизацию, динамический - выявление реальных проблем исполнения. Оптимальная стратегия заключается в их совместном применении: статический анализ для раннего фильтра ошибок, динамический - для проверки корректности и производительности в реальных условиях.

Вывод

В ходе практики я научился применять статический и динамический анализ для выявления ошибок и повышения качества программного обеспечения. Освоил инструменты Flake8, Pylint, Bandit, Checkstyle, SpotBugs и PMD для проверки стиля, архитектуры и безопасности, а также cProfile, memory_profiler, VisualVM и JFR для анализа производительности и утечек памяти. Научился сравнивать отчёты, выявлять ложные срабатывания и пропуски, интегрировать анализаторы в CI/CD и формулировать рекомендации по их применению.